

Atividade 2 - Python e Análise de Dados

Francisco Guthierry de Sousa Cândido

Julho 2026

1 Como identificar e tratar outliers em uma coluna numérica usando desvio padrão ou quartis?

Outliers (valores discrepantes) são valores que se desviam drasticamente do padrão da maioria dos dados. Eles podem ser identificados e tratados por meio de dois métodos: o **método do IQR** (intervalo interquartil), que utiliza os quartis, e o **método do desvio padrão**, que utiliza os conhecimentos da curva de Gauss.

No **método do IQR**, calcula-se o IQR, e a partir dele, os limites superior e inferior:

$$IQR = Q_3 - Q_1$$

$$lim_{sup} = Q_3 + 1,5 \cdot IQR$$

$$lim_{inf} = Q_1 - 1,5 \cdot IQR$$

Após o cálculo dos limites, os outliers podem ser identificados como qualquer valor fora desses limites:

$$x_0 > lim_{sup}$$

$$x_0 < lim_{inf}$$

No **método do desvio padrão** são considerados outliers os valores que excedem três desvios padrão em relação a média:

$$x_0 > \mu + 3 \cdot \sigma$$

$$x_0 < \mu - 3 \cdot \sigma$$

Esse critério é fundamentado na curva de Gauss, segundo a qual 99,7% dos dados se concentram no intervalo de $\mu \pm 3 \cdot \sigma$. Logo, qualquer valor fora desse intervalo é estatisticamente incomum e pode ser considerado outlier.

Após a identificação, os outliers podem ser tratados de diferentes formas, como a remoção (quando justificável), substituição (pela média, mediana ou alguma outra medida) e manutenção (podem ser outliers representativos).

2 Como concatenar vários DataFrames (empilhando linhas ou colunas), mesmo que tenham colunas diferentes?

A concatenação de DataFrames consiste em combinar dois ou mais DataFrames em um único, seja empilhando linhas ou colunas. Em pandas, isso é feito com a função `pd.concat`, que recebe uma lista de DataFrames e o parâmetro `axis`, que define a direção da concatenação (vertical ou horizontal).

Quando `axis=0`, os DataFrames são empilhados verticalmente, ou seja, as linhas de um são adicionadas abaixo do outro. Quando `axis=1`, os DataFrames são concatenados horizontalmente, adicionando colunas lado a lado.

Quando os DataFrames possuem colunas diferentes, o `pd.concat` não descarta as colunas ausentes, ele as mantém e preenche os valores faltantes com `NaN`. Isso ocorre pois a função realiza, por padrão, uma união entre as colunas dos DataFrames, preservando todas as colunas presentes em qualquer um deles.

3 Utilizando pandas, como realizar a leitura de um arquivo CSV em um DataFrame e exibir as primeiras linhas?

A realização da leitura de um arquivo CSV por meio da biblioteca pandas ocorre com a utilização da função `pd.read_csv("arquivo.csv")`. Além do caminho do arquivo, é possível incluir outros parâmetros importantes nessa função, como o `encoding` (define a codificação do arquivo) e o `sep` (define o caractere separador dos campos).

Para exibir as primeiras linhas de um DataFrame é utilizada a função `df.head()`, que por padrão exibe as cinco primeiras linhas. É possível especificar a quantidade desejada, como `df.head(10)`, que exibe as dez primeiras linhas.

Uma outra função importante relacionada a pergunta é a `df.tail()`, que exibe as últimas linhas de um DataFrame. Nessa função, também é possível especificar a quantidade de linhas a serem exibidas.

4 Utilizando pandas, como selecionar uma coluna específica e filtrar linhas em um “DataFrame” com base em uma condição?

A seleção de uma coluna em pandas pode ser feita com `df["nome_coluna"]`, retornando uma `Series`, ou com `df[["nome_coluna"]]`, retornando um `DataFrame`. Para selecionar múltiplas colunas, os nomes são passados em uma lista: `df[["coluna1", "coluna2"]]`. A notação `df.nome_coluna` também é válida, porém não funciona quando o nome da coluna contém espaços ou coincide com um método do pandas.

Para filtrar linhas com base em uma condição, o pandas oferece duas abordagens: passar a condição diretamente nos colchetes, como em `df[df['nucleos_cpu'] > 10]`, ou utilizar o método `df.query("marca == 'HP'")`. Em ambos os casos, os índices originais são preservados no resultado, caso seja necessário reindexar, utiliza-se `.reset_index(drop=True)`.

5 Utilizando pandas, como lidar com valores ausentes (NaN) em um DataFrame?

Valores ausentes (NaN) representam a ausência de um valor em uma determinada observação ou variável de um conjunto de dados. Antes de qualquer tratamento, é necessária uma análise para identificar a quantidade e a origem desses valores. Para isso, o pandas fornece as funções `df.isnull()` e `df.isnull().sum()`, que retornam, respectivamente, uma máscara booleana indicando a posição dos valores ausentes e a contagem deles por coluna.

Após a identificação, o pandas oferece diferentes formas de tratamento. A remoção é feita com `df.dropna()`, que por padrão remove qualquer linha que contenha pelo menos um valor ausente. Caso se deseje remover colunas, utiliza-se o parâmetro `axis=1`. A imputação é feita com `df.fillna()`, que preenche os valores ausentes com um valor especificado, como a média (`df.fillna(df.mean())`), a mediana ou uma constante. A escolha entre remoção e substituição depende do contexto: a remoção é indicada quando a quantidade de valores ausentes é pequena, enquanto a substituição é preferível quando a perda de dados seria prejudicial ao modelo.